

Assignment 4

The CEO has acquired a company working on something similar in hopes that it will make your project sell better. You meet the other team in an awkward meeting, well let's figure this out...

- Your team and the other team in the room are now one team. Assignments are done by, and grades will be awarded to the new team. The two leads will split into two roles: that of team lead and that of technical lead. Technically one lead is enough to demo the project, but they must be able to respond to any question and fix any issue.
- Team lead responsibilities:
 - Communication and task allocation on what needs to be done
 - Presenting the work during the demo
 - Breaking down the grade given to the team among team members (around 12-14)
 - Report creation to demonstrate that everyone worked non negligible amounts. The report must include number of git commits, branches used, total lines changed, diagrams created, and anything else that the team lead thinks is relevant. It also must have a field denoting if the team member did non-negligible amount of work.
- Technical lead responsibilities:
 - Communication and task allocation on how things are done
 - Code reviews (pull requests)
 - Defines the code conventions/style for the team
 - Code quality assurance:
 - readability
 - unit testing
 - adherence to conventions / coding style
 - Deployment to server (once you get one)
- Create a new GitHub repository with the name: UBB-SE-2025-[your subgroup] (e.g. UBB-SE-2025-921-1). Invite the client (imre.zsigmond@ubbcluj.ro) as a collaborator. When adding to this project make sure to only add source files and not files generated by your IDE.
- Technical lead shall create a coding style/convention/standard document that defines how everybody should write their code in the team. There are numerous versions online that you can adopt, alter or create your own from scratch. It must contain at least 15 rules but not more than 30. A rule might be: "Use PascalCase for class names and method names." If your rules are in conflict with stylecop then change stylecop.
- Merge your solution with the other team in the room. Copy your code to the new GIT repo and merge the two structures into one. The new structure must be uniform for all classes. Eg. if a project used the model as if it was a service and the other had a separate service then make sure you have a separate service. Make sure that there is a unified GUI that can access all functionalities and not have two separate apps. If your team's code used mock functions to get data from the other team's application, then use the real functions now. Use this opportunity to purge unused code. While refactoring, make sure that your solution does not contain any business logic in it's UI or it's data access layer. As a reminder, MVVM is just the UI layer.

- I strongly suggest that you start in time and target a few days before the demo as a final deadline. Also, that you use git branches.
- If there are unfinished/buggy functionalities that prevent or make the merge harder, you may finish them. But do it withing reason, do not burn yourself or your team out over this. You may do this for any of the original projects, so if you decide that one of the original projects needs more attention you should allocate your efforts accordingly.
- Switch your data access layer (repository) to use entity framework core. Use "code first" and migrations to create your database. The entire project will use the same database and not two or more different ones. You may still save/load files for anything that is not domain related (e.g. sound/video recordings) but raw sql / stored procedures are gone.
- Put your new repository on a server, and by server, we mean an ASP.NET core web API project that runs separately from the main application. Each team member should be able to run the server locally. This does mean that your application will not connect to the database directly, but through this web API.
- Code readability, testability and coupling rules apply from the previous assignment, as well as the use of StyleCop with the same ruleset. Do not make the application static. Do not have any business logic in the UI or the data access layer.

Deadline: lab 5